Mini Transaction Ledger

Architecture and Internal Workings

Ekramul Alam

MISL Fresher Assessment Project · September 2026

1 Overview

The application is a transaction ledger. Accounts are opened, money is deposited, withdrawn and transferred between them, and every movement of money is recorded as an immutable ledger entry that carries the balance it left behind. A transfer is balanced — one debit and one matching credit that net to zero. Section 8 lists what was deliberately left out.

It is built as three containers started by a single `docker compose up`: an Angular frontend served by nginx, a Spring Boot REST backend, and a PostgreSQL database.

The design goal throughout was that the database should never be able to hold a state that is arithmetically impossible. Balances cannot go negative, an event either lands completely or not at all, two simultaneous withdrawals cannot both spend the same money, and the stored balance can be proved to equal the sum of the entries that produced it.

2 Application architecture

2.1 Frontend and backend interaction

The browser only ever requests paths beginning with `/api/`. It never knows the backend's address. In production, nginx serves the compiled Angular files and forwards `/api/` to the backend container; in development, the Angular dev server does the same job through `proxy.conf.json`.

Because the browser therefore talks to exactly one origin, the browser's same-origin rule is never triggered, and **there is no CORS configuration anywhere in this project**. This was a deliberate architectural choice rather than an omission: a reverse proxy in front of both the static files and the API removes an entire class of configuration and an entire class of bug.

2.2 Backend structure

Figure 1 shows the whole system. The backend is a conventional layered application, but it is **packaged by feature rather than by layer**. There is an `account` package and an `entry` package, each holding its own controller, service, repository and entity, instead of a project-wide `controllers/`, `services/` and `repositories/`.

Layer	Responsibility	Knows nothing about
Controller	bind HTTP, validate, choose the status code	business rules, SQL
Service	business rules, transaction boundaries	HTTP, JSON
Repository	database access	why it is being asked

Table 1: Each layer's responsibility, and what it is deliberately ignorant of.

Two structural decisions are worth drawing attention to.

Repositories are package-private. `AccountRepository` is not declared `public`. The posting feature in the `entry` package therefore *cannot* reach account data directly — it has to ask `AccountService`. The compiler enforces the module boundary, rather than a naming convention

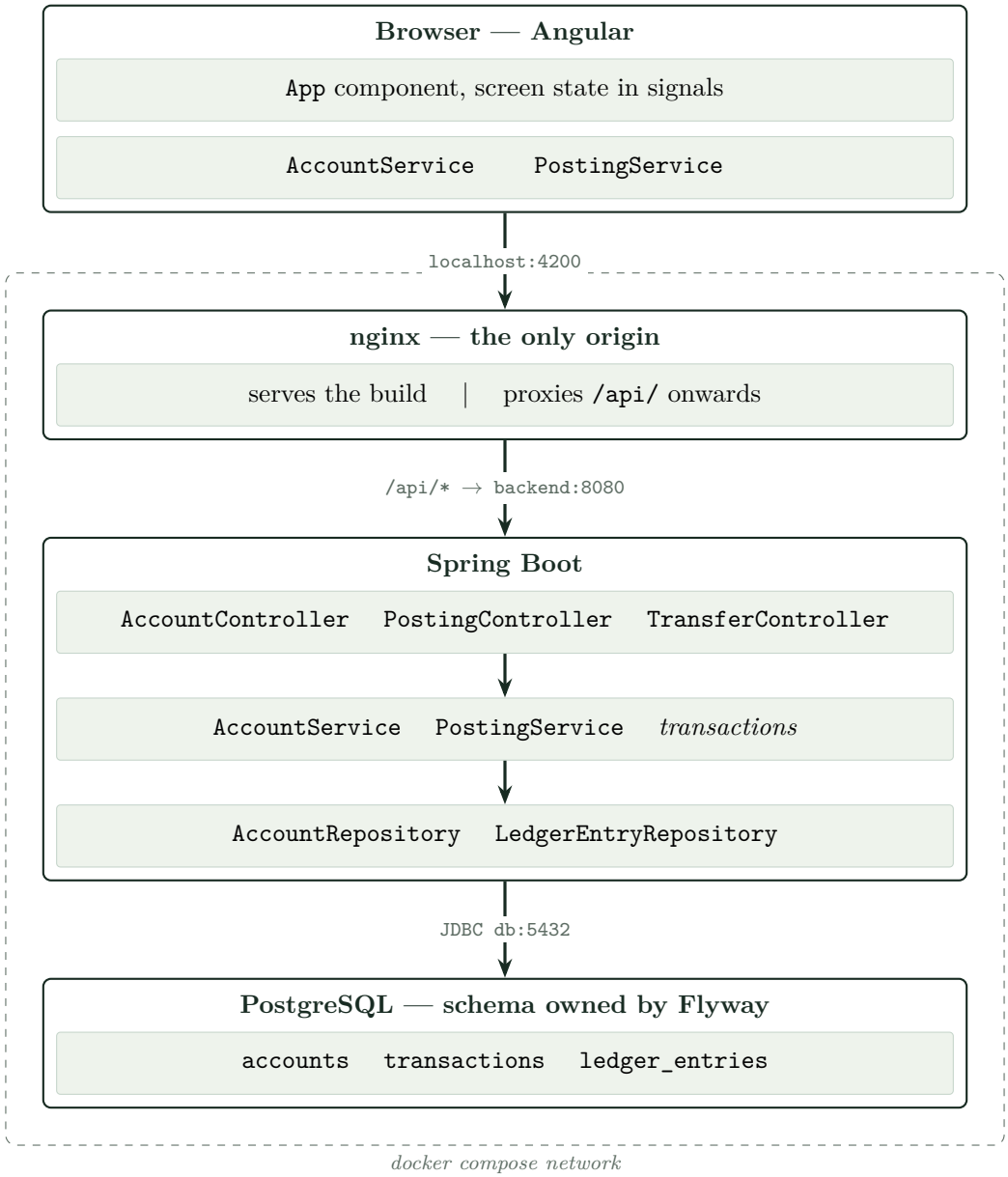

Figure 1: The whole system. The browser requests only `/api/` paths and never learns the backend's address, so every request has one origin and no CORS configuration is needed. Inside the backend each layer talks only to the one beneath it; repositories are package-private, so no feature can reach another feature's data without going through its service. Not drawn, because it sits across all three layers rather than inside one: `GlobalExceptionHandler`, which turns any exception that escapes into an `ApiError` response.

asking politely. A useful consequence is that every balance change flows through one method, so every balance change is locked; there is no second path to get wrong.

DTOs at the edge. Controllers accept and return Java records (`CreateAccountRequest`, `AccountResponse`, `EntryResponse`), never JPA entities. The API contract can then evolve independently of the database schema, and no entity is ever accidentally serialised together with its lazy associations.

3 Key code components

Component	Purpose
`Account`	Entity holding the balance. Has no setters: the balance changes only through `credit()` and `debit()`, and the overdraft rule lives inside `debit()` so no caller can forget it.
`LedgerTransaction`	One financial event — a deposit, withdrawal or transfer. Named to avoid being confused with a database transaction. Carries the client-supplied idempotency key.
`LedgerEntry`	One movement of money on one account. Append-only, no setters. Amount is always positive; a `direction` of `DEBIT` or `CREDIT` carries the sign.
`AccountService`	Opens accounts and moves money. Owns the locking, including the ordered lock acquisition that makes transfers deadlock-free.
`PostingService`	Turns a deposit, withdrawal or transfer into one all-or-nothing unit of work: the event row, the balance change, and the entry rows.
`GlobalExceptionHandler`	A single `@RestControllerAdvice` translating exceptions into HTTP responses, so no service method ever needs a try-catch.
`ApiError`	One error shape for the whole API, so a client never has to guess what a failure looks like.
Flyway migrations	`V1` creates `accounts`; `V2` creates `transactions` and `ledger_entries` with their constraints and indexes.

Table 2: The components that carry the design.

On the frontend there is one page and therefore one component. Two services, `AccountService` and `PostingService`, own every HTTP call; the component holds the screen state in Angular signals and exposes one method per endpoint. There is no router, no state management library and no component library — for a single screen each of those would be ceremony rather than structure.

4 How the API works internally

The clearest way to show the internals is to follow one request the whole way down. Taking `POST /api/transfers`, with the transactional part of it drawn in Figure 2:

1. **nginx** matches the `/api/` prefix and proxies to `backend:8080`, resolving the container by its Compose service name over Docker's internal DNS.
2. `DispatcherServlet`, Spring's front controller, matches the URL and HTTP method to `TransferController.transfer(...)`.

3. **Jackson** deserialises the request body into a `TransferRequest` record.
4. `@Valid` runs Bean Validation *before* the method body executes. If a constraint fails, a `MethodArgumentNotValidException` is thrown and the advice returns `400` listing every rejected field — not merely the first.
5. `PostingService.transfer` is annotated `@Transactional`. Everything from this point is one unit of work. It writes the `transactions` row, then asks `AccountService` to move the money.
6. `AccountService.moveMoney` locks both accounts with `SELECT ... FOR NO KEY UPDATE`, **always taking the lower account id first**, checks that the currencies match, and applies the debit and the credit.
7. Two `ledger_entries` rows are written, each recording the balance it left behind.
8. **Commit.** Hibernate flushes both balance updates and both entries together.
9. The controller returns `201 Created` with the debit entry and the credit entry.

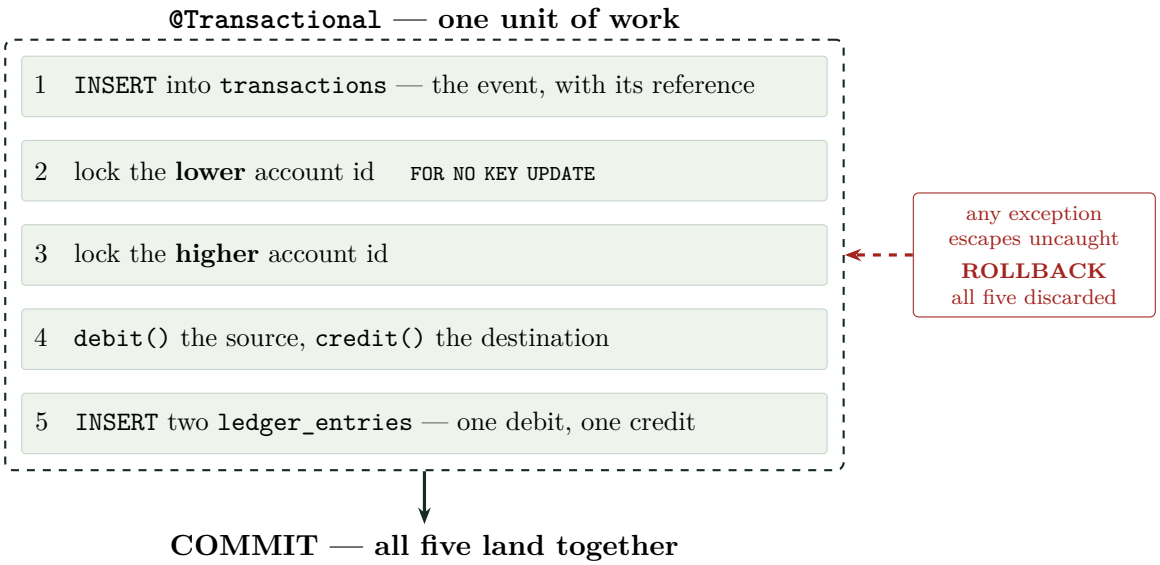

Figure 2: A transfer, from the service's point of view. Because nothing catches the exception, a failure at step 4 — insufficient balance, say — also throws away the event row written at step 1. The ledger never keeps a record of something that did not happen.

4.1 Why there is no try-catch in the service layer

If any step above throws, the exception is allowed to escape. Nothing catches it. Spring sees an unchecked exception leave a `@Transactional` method and rolls the whole transaction back — so the `transactions` row written in step 5 is discarded along with everything else.

This is the reason the service layer contains no defensive try-catch. Catching an exception inside a transactional method would let Spring see a clean return and **commit a half-finished operation**. The ledger would then hold a record of an event that never actually happened. Errors are instead translated once, at the boundary, by `GlobalExceptionHandler`.

That handler deliberately does *not* declare an `@ExceptionHandler(Exception.class)`. A catch-all in a `@RestControllerAdvice` is resolved before Spring's own exception resolvers, so it would intercept the framework's exceptions too — turning malformed JSON, an unknown path and a wrong HTTP method into `500`s instead of the `400`, `404` and `405` they should be.

The cost of that choice is that those framework errors keep Spring Boot's default error body, which has no `message` field. Producing one shape *and* correct framework statuses means extending `ResponseEntityExceptionHandler` and overriding each framework exception in turn.

4.2 Duplicate protection

Every posting request carries a `reference`, a UUID generated by the client and stored under a unique constraint on `transactions`. If a client retries a request whose response was lost, the second insert violates that constraint and the money moves exactly once.

This is a duplicate *guard* rather than full idempotent replay: the retry is answered with `409`, not with the original response.

```
POST /api/accounts/7/deposits
{ "reference": "b4f1...c2", "amount": "1500.75", "description": "salary" }
```

4.3 Money and precision

Amounts are `NUMERIC(19,4)` in PostgreSQL and `BigDecimal` in Java, never `double`. Binary floating point cannot represent 0.1 exactly, and a rounding error invisible in a user interface is unacceptable in a ledger.

Comparisons use `compareTo` rather than `equals`, because two `BigDecimal` values of the same amount but different scale — `10.5` and `10.5000` — are not `equals` to each other, though they do compare as equal.

The frontend receives balances as JSON numbers and only ever displays them. No arithmetic on money happens in the browser, where the only available numeric type is a floating-point double.

5 Correctness under concurrency

This is where most of the engineering went, so it is worth stating explicitly.

Lost updates. Account rows are read with `@Lock(LockModeType.PESSIMISTIC_WRITE)`. On PostgreSQL, Hibernate renders this as `SELECT ... FOR NO KEY UPDATE`, which blocks any other transaction attempting to write the same row. Without it, two simultaneous withdrawals both read the old balance, both decide there is enough money, and both succeed.

Deadlock. A transfer locks two rows, so ordering matters. If $A \to B$ locked A then B while $B \to A$ locked B then A, each would hold what the other needs and neither could finish. `moveMoney` therefore always locks the **lower account id first**, whichever direction the money is going, so both transfers queue for the same row: one waits and the other proceeds.

The stored balance. `accounts.balance` is a materialized derived aggregate. The entries are canonical; the column exists only so that reading a balance does not require scanning an account's whole history. Three safeguards keep it honest: only `credit()` and `debit()` write it, it is written in the same transaction as the entry that causes it, and a test replays the entries and asserts they sum to it.

Evidence. Both concurrency tests were run once with their safeguard deliberately removed, to confirm that they can actually fail:

Neither concurrency test is annotated `@Transactional`. A test transaction would hold the very rows the worker threads are competing for, and the test would hang rather than fail; concurrency can only be observed against committed data.

Test	Safeguard removed	Result
Ten simultaneous withdrawals of the full balance	`PESSIMISTIC_WRITE`	All ten succeeded. 1000 withdrawn against a balance of 100, and the balance still read zero.
Ten transfers, five in each direction	lowest-id-first ordering	Nine of ten died with SQLSTATE `40P01`, *deadlock detected*.

Table 3: Control experiments. A passing test proves nothing until it has been seen to fail.

6 Docker setup

`docker-compose.yml` defines three services.

Service	Base image	Notes
`db`	`postgres:16`	Named volume, so data survives `docker compose down` and container rebuilds. Host port 5433 avoids clashing with a locally installed PostgreSQL.
`backend`	built multi-stage	Waits for the database to be *healthy*, not merely started.
`frontend`	built multi-stage	nginx serving the compiled application and proxying `/api` to the backend.

Table 4: The three services and why each is configured as it is.

Multi-stage builds. The backend compiles in a full JDK image and runs in a JRE image, so no compiler, no Maven and no source code reach the runtime image; it also runs as a non-root user, so a compromised application cannot own its container. The frontend builds with Node and runs on nginx, so neither Node nor `node_modules` ship.

Layer caching. In both Dockerfiles the dependency manifest is copied and resolved *before* the source is copied. Docker caches each step, so dependencies are re-downloaded only when `pom.xml` or `package-lock.json` changes, not on every code edit.

Healthchecks. "The container is running" is not the same statement as "PostgreSQL is ready to accept queries". The database declares a `pg_isready` healthcheck, and the backend depends on `service_healthy`, so it never starts against a database that is still initialising.

Configuration. Connection details come from environment variables with local defaults, so a reviewer can start the project without creating any files. Nothing is hard-coded, and a real deployment overrides them with a `.env` file.

Single-page routing. nginx serves `try_files $uri $uri/ /index.html`, so refreshing the browser on any path returns the application rather than a 404.

7 Testing

- **Context test** — starting Spring proves every bean wires and, because `ddl-auto=validate`, that the entities still match the migrated schema.

- **Concurrent withdrawal** — ten threads, one balance, exactly one winner, and the nine rolled-back attempts leave no `transactions` rows behind.
- **Concurrent transfer** — ten transfers in both directions at once; none deadlock and the money is conserved.
- **Reconciliation** — replaying an account's entries reproduces its stored balance.
- **Frontend** — the component renders, and the UUID fallback used outside a secure browsing context produces valid v4 references.

8 Scope

Deliberately out of scope for this assessment, each a decision rather than an oversight:

- **Authentication and authorization** — the brief places JWT under Problem 2. Production would need an owner on each account, an ownership check on the debit side of a transfer, and an actor recorded on every transaction.
- **Full double-entry** — a transfer is balanced, but a deposit has a single leg. A general ledger gives it a settlement counterparty, which makes *every entry, signed by direction, sums to zero* a testable invariant.
- **Pagination, lock timeouts, a currency reference table, and CI.**

The full source, including the README with setup instructions, is in the accompanying repository.